

tutorialverse: Shared Infrastructure for Interactive R Tutorials

Shaurita D. Hutchins 
University of Alabama at Birmingham

Samuel Bharti 
University of Alabama at Birmingham

Executive Summary

R has extensive educational material in package documentation, vignettes, books, workshops, Carpentries lessons, university courses, and community websites. Interactive tutorials offer immediate practice and feedback, but authoring conventions, metadata, validation, discovery, maintenance, and deployment remain largely project-specific.

tutorialverse will create shared infrastructure for reusable interactive R learning resources. It will deliver an R package for scaffolding, validating, documenting, and publishing tutorials; a common metadata specification; reusable templates and testing workflows; a searchable registry of independently maintained tutorials; a reproducible submission and community-review process; and a static-hosting prototype for compatible tutorials.

Four reference tutorials will exercise this infrastructure: reproducible environments with **renv**, package development with **usethis** and **devtools**, a Bioconductor workflow using **DESeq2**, and an interactive companion to an introductory Carpentries R lesson. These are test cases for the infrastructure, not the primary product. The 12-month project will build on **learnr**, Quarto, R Markdown, existing training resources, and hosting systems. The requested budget remains to be finalized against the five milestones.

Signatories and Community Support

Shaurita D. Hutchins, co-lead for educational infrastructure and community development, is a PhD candidate in genetics, genomics and bioinformatics at the University of Alabama at Birmingham. She will lead the tutorial specification, educational design, accessibility framework, reference tutorials, and community contribution model, drawing on experience in computational biology, R/Bioconductor workflows, reproducible research, open-source training, and Carpentries instruction. Her relevant work includes [Prioritizing Candidate Genetic Variants Using GTEx, HuBMAP, and IDG](#), a CFDE training module with browser-based coding activities and knowledge checks.

Samuel Bharti, co-lead for software architecture and deployment infrastructure, is a PhD candidate at the University of Alabama at Birmingham. He will lead package architecture, validation and testing, deployment tooling, and integration with Shiny and browser-based R. His relevant work includes [r-for-beginners](#), an adaptation of Software Carpentry lessons with slides; the R packages **bioclients**, **biohttp**, and **biobouncer**; and package documentation published through GitHub Actions and GitHub Pages.

The co-leads will jointly develop the package API, metadata specification, registry architecture, governance, and reference implementations. Before finalizing the specification, they will seek feedback from **learnr** contributors, Carpentries instructors and maintainers, Bioconductor education contributors, package maintainers, and university and community instructors. Consultation is planned; letters of support and documented feedback will be included only when received.

The Problem

R does not lack educational content. It needs shared infrastructure connecting that content. Five recurring problems limit reuse:

1. **Fragmented discovery.** Learners and instructors search across documentation, course repositories, and community sites without consistent fields for packages taught, prerequisites, objectives, difficulty, duration, or maintenance status.
2. **Fragile tutorials.** Changes to APIs, datasets, links, dependencies, and rendering tools can break exercises without notifying maintainers. Authors need automated checks before learners encounter failures.
3. **Costly conversion.** Adapting existing lessons repeatedly requires decisions about exercises, structure, metadata, accessibility, dependencies, testing, and publication. Common scaffolding can reduce that work while preserving source ownership.
4. **Deployment overhead.** Local installation and server administration can impede access. Browser execution offers another route, but package and exercise compatibility must be tested for each tutorial.
5. **Inconsistent maintenance.** Tutorials need versioning, ownership, dependency declarations, validation dates, and archival policies that remain understandable across projects.

Existing tools address parts of this problem. `learnr` supports interactive R tutorials with exercises and assessments (Aden-Buie et al. 2025). Quarto Live uses webR to execute R exercises in HTML pages served by static hosts (Stagg 2026). Carpentries lessons, Bioconductor training, and package vignettes provide material to build upon. tutorialverse will connect these resources through common metadata, validation, discovery, and review, with explicit execution requirements. It will not create a learning-management system or take over tutorial hosting.

The Proposal

Milestone 1: Specification and authoring tools

Publish a version 1.0 metadata specification and an R package with 3–4 reusable templates, authoring conventions, schema validation, and accessibility guidance. Metadata will describe identity and version, title and description, maintainers, license, topics and packages taught, level, prerequisites, duration, objectives, language, accessibility information, dependencies, execution requirements, tutorial engine, deployment modes, canonical repository and tutorial URL, last validation date, and maintenance status.

Provisional interfaces are `create_tutorial()`, `tutorial_metadata()`, `check_tutorial()`, `build_tutorial()`, and `register_tutorial()`. Community review and prototyping will refine the API. Templates will separate instructional content, exercise setup, and metadata, and document how to adapt existing materials while retaining attribution and compatible licenses.

Milestone 2: Validation and continuous integration

Implement checks for required metadata, dependency availability and declared versions, executable example code, links, files, exercise identifiers, rendering, basic accessibility, and compatibility with declared deployment targets. A reusable GitHub Actions workflow will produce machine-readable validation reports and actionable errors identifying the failing field or exercise. Automated accessibility checks will complement human review of keyboard use and instructional clarity.

External repositories will be able to adopt validation independently of registry submission. Package checks will run on Linux, macOS, and Windows. Tutorial checks will record the tested environment and date so a successful metadata check is distinguishable from a successful execution test.

Milestone 3: Distribution and static deployment

Document publishing adapters for local `learnr` execution and Shiny-compatible hosting. In parallel, prototype browser-only deployment using webR, Quarto Live, and browser-based Shiny where appropriate. Bharti will lead compatibility testing, including whether R-universe WebAssembly packages can support the reference workflows. This work will assess a concrete deployment path without assuming that `learnr` features translate directly.

Deliver a static-hosting prototype, a reusable GitHub Pages workflow, compatibility tests, and documented supported features, limitations, and server-backed fallbacks. Test compatible examples in Chrome, Firefox, and Safari, and report package download sizes and execution constraints. Publish at least one static reference tutorial if feasible. **renv**, package-development, and Bioconductor exercises may require capabilities unavailable in a browser; their declared execution modes will reflect the results.

Milestone 4: Registry and community review

Build a searchable registry that indexes metadata and links to canonical repositories and deployments. Tutorials retain their independent ownership. The initial architecture will use an R package repository, reusable template sources, and a registry repository with one YAML entry per tutorial. The registry will publish a website and `index.json`; a proposed `tutorial_search()` function will return matching entries as an R data frame.

A structured pull request will submit or nominate a tutorial. Automated validation will precede human review of instructional clarity, accessibility, scope, licensing, and maintenance expectations. Review criteria will draw on existing open lesson-review practices ([The Carpentries 2022](#)). Submission documentation will distinguish author submissions from nominations and provide correction and removal processes.

Users will filter by topic, package, level, objective, duration, engine, deployment mode, and maintenance status. Scheduled builds and link checks will flag broken or potentially stale entries under a published policy. Link availability will not be presented as proof that exercises still work. Deliverables include the website, nomination and review workflows, governance documentation, and machine-readable index.

Milestone 5: Reference implementations

Complete four tutorials: **renv** for reproducible environments; **usethis/devtools** for package development; **DESeq2** for an introductory Bioconductor workflow; and a **Carpentries companion** aligned with an introductory R lesson. Bharti's r-for-beginners adaptation and existing Bioconductor lessons provide starting points, subject to their licenses and attribution requirements.

Each tutorial will test metadata, validation, accessibility, dependencies, and deployment requirements. Development will begin early enough to inform the specification and testing framework. The final set will demonstrate how existing educational content can participate without being rewritten or centrally maintained.

Project Plan

Schedule and working practices

Period	Work and acceptance point
Months 1–2	Audit formats and metadata; consult communities; publish the draft specification; establish repositories, governance, and the first template. Review the draft before fixing the schema.
Months 3–5	Implement scaffolding, metadata parsing, validation, CI, and developer documentation. Test representative tutorials and demonstrate useful errors for invalid submissions.
Months 6–8	Document standard hosting; prototype browser deployment; test dependencies and features; publish compatibility results and a GitHub Pages workflow. Demonstrate a static reference tutorial if feasible.
Months 9–10	Implement registry ingestion, search, submissions, and review; pilot external entries; conduct community usability testing.
Months 11–12	Complete and validate four reference tutorials; resolve high-priority feedback; release package, specification, and registry version 1.0 with contribution and maintenance documentation.

Development will use public GitHub issues, milestones, pull requests, code review, automated tests, and documented releases. Monthly progress notes and milestone reports will make delivery visible. Community feedback will be sought during specification, prototype, and registry review. The package will use standard R tooling, including `testthat`, `pkgdown`, and **renv**, with R-universe distribution where appropriate.

The co-leads will seek input from educators, package maintainers, tutorial authors, accessibility reviewers, and **learnr** or Shiny contributors. Code will use MIT licensing; newly authored tutorial content and documentation will use CC BY 4.0 where compatible with source-material obligations. Adaptations will preserve required attribution and licensing. Release announcements and author guidance will be shared with R education communities.

Budget and funding

Requested budget: [TO BE FINALIZED]. Funding will support development, specification, testing, documentation, deployment prototyping, registry implementation, accessibility, consultation, and reference tutorials. Before submission, the budget will assign developer hours, rates, and amounts to each of the five milestones and distinguish each co-lead’s funded effort. If funding is reduced, specification, scaffolding, and validation provide a useful standalone first phase.

Risks and sustainability

Deployment compatibility: static execution is a compatibility target, not a requirement for every tutorial. Publish supported features and fallback guidance; retain valid server-backed entries. If the prototype cannot support a reference tutorial, report that result rather than relabeling precomputed output as an executed analysis.

Registry maintenance: retain decentralized ownership, canonical links, maintainers, validation dates, and status fields. Scheduled checks and a documented archival process will expose stale entries. Reusable templates, contributor documentation, and governance will reduce dependence on the two leads.

Scope and adoption: keep the four reference tutorials as infrastructure test cases; seek additional examples through contributions and nominations. Publish the specification early and revise it through external review. Existing tools will supply execution and hosting; tutorialverse will focus on metadata, scaffolding, validation, publication, discovery, and review.

Success

The project is complete when the specification, package, and registry reach version 1.0; scaffolding and validation are documented; the registry ingests independently maintained tutorials; four reference tutorials pass their declared validation checks; and deployment compatibility, governance, contribution, and maintenance processes are published.

Measure	Target
Metadata specification, public R package, registry	Version 1.0 of each
Reusable tutorial templates	3–4
Reference tutorials validated and tested in CI	All four
External tutorials indexed during the pilot	At least three
External educators, maintainers, or contributors reviewing the work	At least three
Reusable GitHub Actions validation workflow	At least one
Static deployment prototype and compatibility report	At least one prototype; deployed reference tutorial if feasible
Public governance, contribution, and maintenance documentation	Published

Validation success will mean passing the documented checks, not proof of complete instructional or accessibility coverage. During the grant, track external submissions, review participation, and usability feedback. After release, monitor packages covered, new entries, validation freshness, and broken links to identify maintenance needs and gaps in educational coverage.

Future work, outside the initial funded scope, may include additional tutorial engines, multilingual metadata, richer accessibility checks, instructor collections, and integration with pkgdown, R-universe, or other educational catalogs. The first release will concentrate on reusable infrastructure that communities can adopt and maintain independently.

References

- Aden-Buie, Garrick, Barret Schloerke, J. J. Allaire, and Alexander Rossell Hayes. 2025. *Learnr: Interactive Tutorials for r*. Released. <https://CRAN.R-project.org/package=learnr>.
- Stagg, George. 2026. *Quarto-Live: WebAssembly Powered Code Blocks and Exercises for r and Python in Quarto Documents*. Released. <https://github.com/r-wasm/quarto-live>.
- The Carpentries. 2022. *Piloting Open Peer Review of Lessons*. <https://carpentries.org/blog/2022/02/carpentries-lab-lesson-review-pilot/>.